



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: 二叉查找树的编程与实现

一、实验目的

掌握二叉查找树的基本概念及其基本操作的实现

二、实验原理

参照课本 p.73-80, 及 Figure4.16-4.25

二叉查找树又称二叉排序树, 是一种具有有序性质的二叉树。对于树中的任意一个结点, 其左子树中所有结点的值都小于该结点的值, 右子树中所有结点的值都大于该结点的值。因此, 二叉查找树可以用于较高效地完成数据的查找和插入操作。

本实验采用链式存储结构实现二叉查找树。每个结点由三部分组成: 数据域、左孩子指针和右孩子指针。其中, 数据域用于存放整数数据, 左孩子指针指向左子树, 右孩子指针指向右子树。通过指针之间的连接, 可以形成完整的二叉查找树结构。

创建二叉查找树时, 程序按照输入顺序依次插入整数。若当前树为空, 则新数据作为根结点; 若新数据小于当前结点, 则继续在左子树中查找插入位置; 若新数据大于当前结点, 则继续在右子树中查找插入位置。直到遇到空位置时, 创建新结点并完成插入, 从而保证插入后的树仍然满足二叉查找树的性质。

查找操作同样利用二叉查找树的有序性。查找时先将关键字 **key** 与当前结点进行比较, 若相等则查找成功; 若 **key** 小于当前结点值, 则在左子树中继续查找; 若 **key** 大于当前结点值, 则在右子树中继续查找; 若最终查找到空结点, 则说明该关键字不存在。实验要求中也明确要求实现二叉查找树的定义、查找和插入操作, 并在查找不到指定 **key** 时将其插入, 使其仍保持二叉排序树性质。

此外, 由于二叉查找树具有“左小右大”的特点, 因此对其进行中序遍历时, 可以得到按递增顺序排列的数据; 若采用反向中序遍历, 即先访问右子树, 再访问根结点, 最后访问左子树, 则可以得到按递减顺序排列的数据。通过这两种遍历方式, 可以验证二叉查找树的有序性。

三、实验内容

输入数据的第一行为一个正整数 **T**, 表示测试数据的组数, 即共 **T** 组测试数据。每组测试数据的第一行输入正整数 $n(5 \leq n \leq 20)$, 第二行输入 **n** 个整数, 要求依次完成以下工作:

- 1) 以这 **n** 个整数生成一棵用链式存储结构存储的二叉排序树
- 2) 按递增顺序输出该二叉排序树中所有数
- 3) 输入一个整数 **key**, 对该二叉排序树进行查找, 若在该二叉排序树中存在这个整数 **key**, 则输出 **find**, 否则输出 **not find**
- 4) 输入一个整数 **key**, 若该二叉排序树中不存在这个整数 **key**, 则将 **key** 插入到该二叉排序树中, 使插入后仍为原性质的二叉排序树; 否则不必插入
- 5) 在 4) 的基础上, 按递减顺序输出该二叉排序树中的所有数

四、实验要求

- 1) 实现二叉查找树的定义
- 2) 实现二叉查找树的查找、插入操作

五、实验源程序

```
#include <stdio.h>
#include <stdlib.h>

/* 二叉排序树结点定义 */
typedef struct BSTNode
{
    int data;
    struct BSTNode *left;
    struct BSTNode *right;
} BSTNode, *BSTree;

/* 队列结点定义，用于按层输出二叉排序树结构 */
typedef struct QueueNode
{
    BSTree treeNode;
    struct QueueNode *next;
} QueueNode;

/* 链式队列定义 */
typedef struct LinkQueue
{
    QueueNode *front;
    QueueNode *rear;
} LinkQueue;

/* 函数声明 */
BSTree NewTreeNode(int data);
BSTree InsertBST(BSTree root, int data);
int SearchBST(BSTree root, int key);
BSTree InsertIfAbsent(BSTree root, int key);

void PrintIncreasing(BSTree root);
void PrintDecreasing(BSTree root);
void PrintLevelOrder(BSTree root);
void DestroyTree(BSTree root);

LinkQueue *InitQueue(void);
int QueueIsEmpty(LinkQueue *Q);
void EnQueue(LinkQueue *Q, BSTree node);
BSTree DeQueue(LinkQueue *Q);
void DestroyQueue(LinkQueue *Q);
```

```

int main()
{
    int T;

    printf("请输入测试数据组数 T: ");
    scanf("%d", &T);

    for (int group = 1; group <= T; group++)
    {
        int n;
        int value;
        int key;
        BSTree root = NULL;

        printf("\n 第 %d 组数据\n", group);

        printf("请输入结点个数 n: ");
        scanf("%d", &n);

        if (n < 5 || n > 20)
        {
            printf("输入的 n 不符合要求, n 应满足 5 <= n <= 20。 \n");
            continue;
        }

        printf("请输入 %d 个整数: ", n);

        for (int i = 0; i < n; i++)
        {
            scanf("%d", &value);
            root = InsertBST(root, value);
        }

        printf("\n 二叉排序树的层次结构为: \n");
        PrintLevelOrder(root);

        printf("\n 递增顺序输出: ");
        PrintIncreasing(root);
        printf("\n");

        printf("请输入要查找的 key: ");
        scanf("%d", &key);

        if (SearchBST(root, key))
    }
}
    
```

```

        {
            printf("find\n");
        }
        else
        {
            printf("not find\n");
        }

        printf("请输入要插入的 key: ");
        scanf("%d", &key);

        if (SearchBST(root, key))
        {
            printf("该结点已经存在, 不执行插入操作.\n");
        }
        else
        {
            root = InsertBST(root, key);
            printf("插入成功.\n");
        }

        printf("插入后的层次结构为: \n");
        PrintLevelOrder(root);

        printf("\n 递减顺序输出: ");
        PrintDecreasing(root);
        printf("\n");

        DestroyTree(root);
    }

    return 0;
}

/* 创建新结点 */
BSTree NewTreeNode(int data)
{
    BSTree node = (BSTree)malloc(sizeof(BSTNode));

    if (node == NULL)
    {
        printf("内存申请失败! \n");
        exit(1);
    }
}

```

```

        node->data = data;
        node->left = NULL;
        node->right = NULL;

        return node;
    }

    /* 向二叉排序树中插入结点 */
    BSTree InsertBST(BSTree root, int data)
    {
        if (root == NULL)
        {
            return NewTreeNode(data);
        }

        if (data < root->data)
        {
            root->left = InsertBST(root->left, data);
        }
        else if (data > root->data)
        {
            root->right = InsertBST(root->right, data);
        }
        else
        {
            /* 重复数据不插入 */
            return root;
        }

        return root;
    }

    /* 查找指定 key 是否存在 */
    int SearchBST(BSTree root, int key)
    {
        if (root == NULL)
        {
            return 0;
        }

        if (key == root->data)
        {
            return 1;
        }
    }

```

```

    }
    else if (key < root->data)
    {
        return SearchBST(root->left, key);
    }
    else
    {
        return SearchBST(root->right, key);
    }
}

```

/* 如果 key 不存在，则插入 */

```

BSTree InsertIfAbsent(BSTree root, int key)
{
    if (!SearchBST(root, key))
    {
        root = InsertBST(root, key);
    }

    return root;
}

```

/* 中序遍历：递增输出 */

```

void PrintIncreasing(BSTree root)
{
    if (root == NULL)
    {
        return;
    }

    PrintIncreasing(root->left);
    printf("%d ", root->data);
    PrintIncreasing(root->right);
}

```

/* 反向中序遍历：递减输出 */

```

void PrintDecreasing(BSTree root)
{
    if (root == NULL)
    {
        return;
    }

    PrintDecreasing(root->right);
}

```

```

        printf("%d ", root->data);
        PrintDecreasing(root->left);
    }

/* 初始化链式队列 */
LinkQueue *InitQueue(void)
{
    LinkQueue *Q = (LinkQueue *)malloc(sizeof(LinkQueue));

    if (Q == NULL)
    {
        printf("队列创建失败! \n");
        exit(1);
    }

    Q->front = NULL;
    Q->rear = NULL;

    return Q;
}

/* 判断队列是否为空 */
int QueueIsEmpty(LinkQueue *Q)
{
    return Q->front == NULL;
}

/* 入队 */
void EnQueue(LinkQueue *Q, BSTree node)
{
    QueueNode *newNode = (QueueNode *)malloc(sizeof(QueueNode));

    if (newNode == NULL)
    {
        printf("队列结点申请失败! \n");
        exit(1);
    }

    newNode->treeNode = node;
    newNode->next = NULL;

    if (Q->rear == NULL)
    {
        Q->front = newNode;
    }
}

```

```

        Q->rear = newNode;
    }
    else
    {
        Q->rear->next = newNode;
        Q->rear = newNode;
    }
}

/* 出队 */
BSTree DeQueue(LinkQueue *Q)
{
    QueueNode *temp;
    BSTree node;

    if (QueueIsEmpty(Q))
    {
        return NULL;
    }

    temp = Q->front;
    node = temp->treeNode;

    Q->front = Q->front->next;

    if (Q->front == NULL)
    {
        Q->rear = NULL;
    }

    free(temp);

    return node;
}

/* 释放队列 */
void DestroyQueue(LinkQueue *Q)
{
    while (!QueueIsEmpty(Q))
    {
        DeQueue(Q);
    }

    free(Q);
}

```

```

}

/* 按层次输出二叉排序树结构 */
void PrintLevelOrder(BSTree root)
{
    if (root == NULL)
    {
        printf("当前二叉排序树为空。\\n");
        return;
    }

    LinkQueue *Q = InitQueue();

    EnQueue(Q, root);

    int level = 1;
    int currentCount = 1;
    int nextCount = 0;

    printf("第%d 层: ", level);

    while (!QueueIsEmpty(Q))
    {
        BSTree current = DeQueue(Q);

        printf("%d ", current->data);
        currentCount--;

        if (current->left != NULL)
        {
            EnQueue(Q, current->left);
            nextCount++;
        }

        if (current->right != NULL)
        {
            EnQueue(Q, current->right);
            nextCount++;
        }

        if (currentCount == 0 && nextCount > 0)
        {
            level++;
            printf("\\n 第%d 层: ", level);
        }
    }
}
    
```

```

        currentCount = nextCount;
        nextCount = 0;
    }
}

printf("\n");

DestroyQueue(Q);
}

/* 释放二叉排序树 */
void DestroyTree(BSTree root)
{
    if (root != NULL)
    {
        DestroyTree(root->left);
        DestroyTree(root->right);
        free(root);
    }
}

```

六、实验结果

测试两种不同的二叉查找树（7 结点），分别尝试查找二叉树中存在/不存在的整数，以及分别尝试插入二叉树中存在/不存在的整数，运行结果如下

```

PS D:\TEX\IEEE> cd "d:\Epan\Study\数据结构\源程序\" ; if ($?) { gcc Shiyang4.c -o Shiyang4 } ; if ($?) { .\Shiyang4 }
请输入测试数据组数 T: 2

第 1 组数据
请输入结点个数 n: 7
请输入 7 个整数: 4 2 6 1 3 5 7

二叉排序树的层次结构为:
第1层: 4
第2层: 2 6
第3层: 1 3 5 7

递增顺序输出: 1 2 3 4 5 6 7
请输入要查找的 key: 3
find
请输入要插入的 key: 9
插入成功。
插入后的层次结构为:
第1层: 4
第2层: 2 6
第3层: 1 3 5 7
第4层: 9

递减顺序输出: 9 7 6 5 4 3 2 1

第 2 组数据
请输入结点个数 n: 7
请输入 7 个整数: 10 6 14 1 19 8 13

二叉排序树的层次结构为:
第1层: 10
第2层: 6 14
第3层: 1 8 13 19

递增顺序输出: 1 6 8 10 13 14 19
请输入要查找的 key: 20
not find
请输入要插入的 key: 10
该结点已经存在, 不执行插入操作。
插入后的层次结构为:
第1层: 10
第2层: 6 14
第3层: 1 8 13 19

递减顺序输出: 19 14 13 10 8 6 1

```

七、实验心得

通过本次实验，我完成了二叉查找树的建立、查找、插入以及递增、递减输出等操作，对二叉查找树的基本性质有了更深入的理解。二叉查找树的特点是左子树结点值小于根结点，右子树结点值大于根结点，因此在查找和插入时可以根据关键字大小选择进入左子树或右子树，从而提高操作效率。

在程序实现过程中，我认识到二叉查找树的结构与数据输入顺序有关。如果输入数据接近有序，树可能会退化成类似链表的结构，导致层数增加、查找效率降低。同时，通过中序遍历可以得到递增序列，反向中序遍历可以得到递减序列，这进一步体现了二叉查找树的有序性。

本次实验也加深了我对递归、指针和动态内存分配的理解，提高了我将数据结构理论转化为具体程序的能力。